

Lecture 13-15 - Module 4.5 Regularization

(applications on linear models)

COMP 551 Applied machine learning

Yue Li

Assistant Professor

School of Computer Science

McGill University

Feb 20, 25, 27, 2025

Outline

Objectives

Revisit model complexity and overfitting

Ridge regression

Bayesian interpretation of regularized regression

L1-regularized regression

- Algorithm of fitting L_1 regularized regression model

- Group LASSO

- Elastic net

Bias-variance trade-off

Summary

Learning objectives

Understanding the basic ideas of

- Model complexity and overfitting (revisit)
- Regularization penalty (L_2 & L_1 norm)
- Ridge regression with L_2 regularization and Bayesian prior
- LASSO model with L_1 regularization and Bayesian prior
- Maximum a posteriori (MAP) estimation
- Bias and variance trade-off

Outline

Objectives

Revisit model complexity and overfitting

Ridge regression

Bayesian interpretation of regularized regression

L1-regularized regression

- Algorithm of fitting L_1 regularized regression model

- Group LASSO

- Elastic net

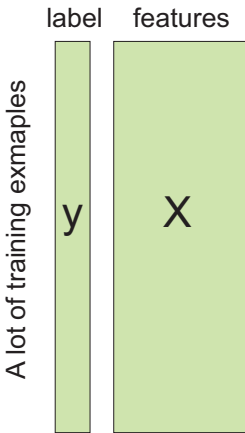
Bias-variance trade-off

Summary

Large P, small N problem

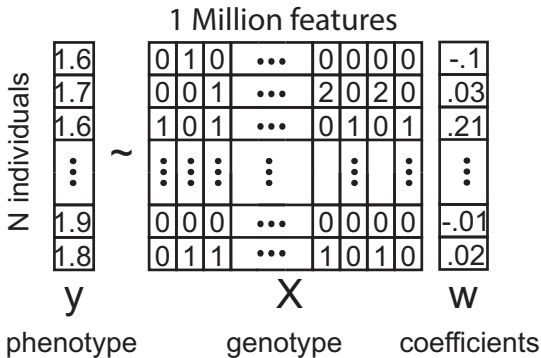
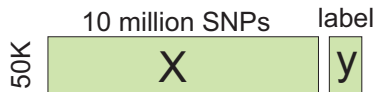
Standard benchmark
data (e.g., iris flowers)

$$N > P$$



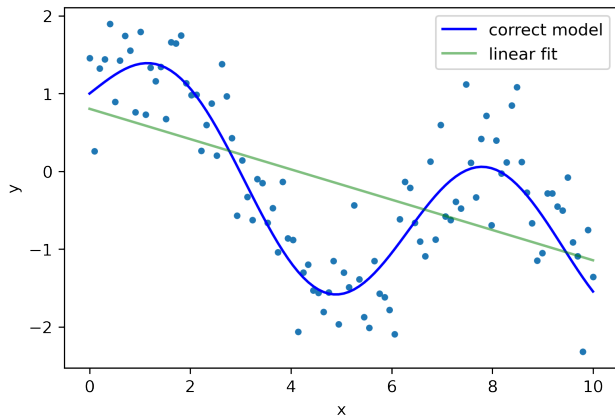
Real-world data
(e.g., population genetic data)

$$P \gg N$$



Even with small number of features, high model expressiveness is desired

It is obvious that our attempt to model y as a linear function of $\hat{y} = w_0 + xw_1$ would produce a bad fit.



Fitting non-linear data by transforming the input features

Recall from Module 4.1 Linear Regression that we can fit non-linear data by transforming the input feature x with a (non-linear) **basis function** $\phi(x)$.

The multiple linear regression operates on the basis-transformed features:

$$\hat{y} = \sum_d w_d \phi_d(x) = \Phi(x) \mathbf{w}$$

We then simply replace all of the occurrences of x with $\Phi(x)$ in the OLS solution:

$$\hat{\mathbf{w}} = (\Phi(x)^T \Phi(x))^{-1} \Phi(x)^T \mathbf{y}, \quad \text{where}$$

$$\Phi(\mathbf{x}) = \begin{bmatrix} \phi_1(x^{(1)}) & \phi_2(x^{(1)}) & \dots & \phi_M(x^{(1)}) \\ \phi_1(x^{(2)}) & \phi_2(x^{(2)}) & \dots & \phi_M(x^{(2)}) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_1(x^{(N)}) & \phi_2(x^{(N)}) & \dots & \phi_M(x^{(N)}) \end{bmatrix}$$

Nonlinear basis functions (Recall from Module 4.1 Linear Regression)

There are many nonlinear basis functions. Using scalar input $x \in \mathbb{R}$ as an example,

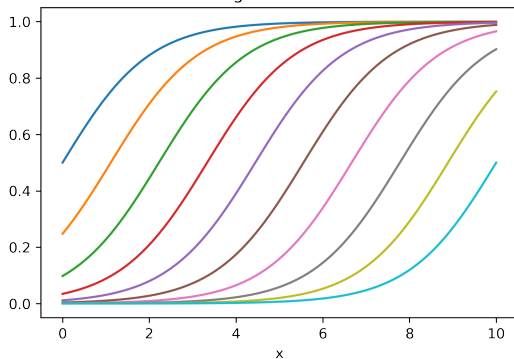
Gaussian bases



$$\phi_d(x) = \exp\left(-\frac{(x - \mu_d)^2}{s^2}\right)$$

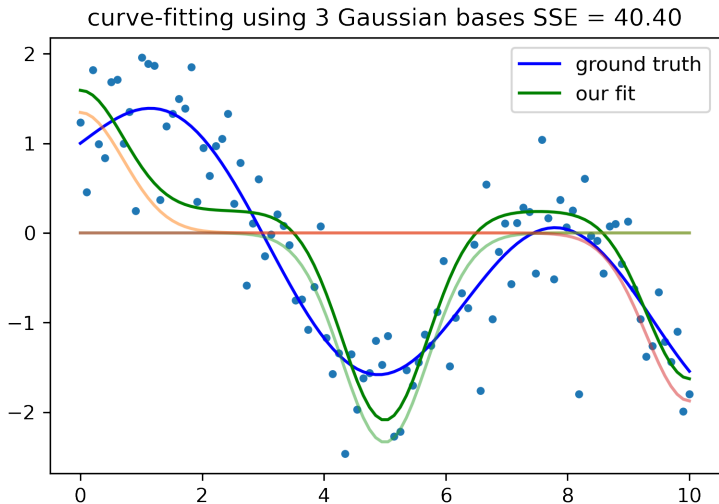
where each type of basis function has a different mean $\mu_d \in [0, 10]$ and $s = 1$.

Sigmoid bases



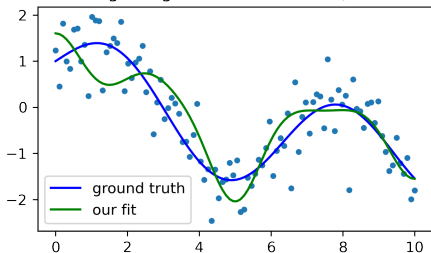
$$\phi_d(x) = \frac{1}{1 + \exp\left(-\frac{(x - \mu_d)^2}{s^2}\right)}$$

Linear regression with Gaussian nonlinear basis features ($\hat{y} = \sum_d w_d \phi_d(\mathbf{x})$)

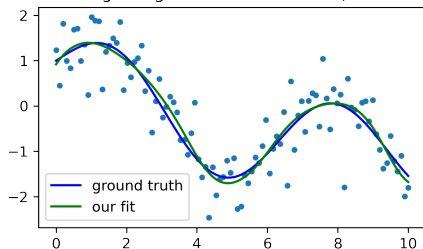


Increasing the number of Gaussian bases to 5, 10, 50, 200 (colab)

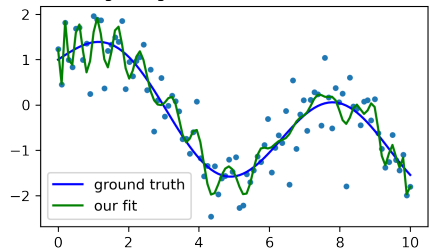
Curve-fitting using 5 Gaussian bases (SSE = 36.18)



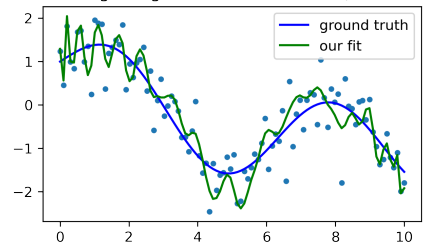
Curve-fitting using 10 Gaussian bases (SSE = 23.99)



Curve-fitting using 50 Gaussian bases (SSE = 15.81)

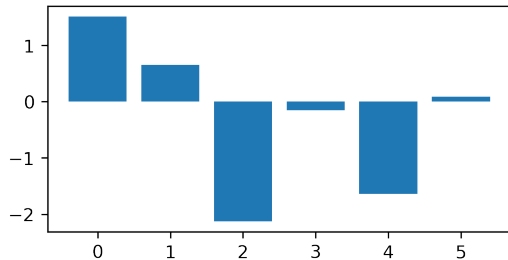


Curve-fitting using 200 Gaussian bases (SSE = 19.25)

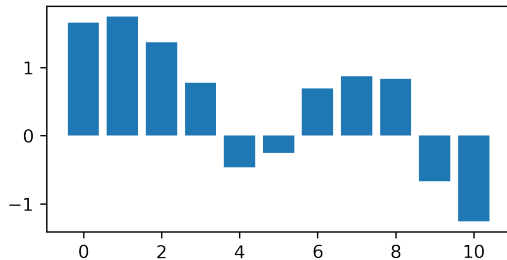


High number of features led to very large weights!

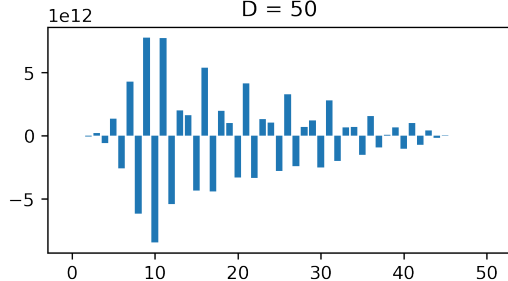
D = 5



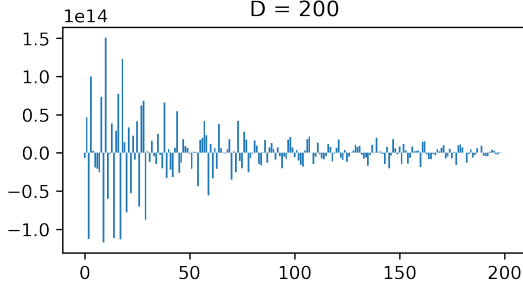
D = 10



D = 50



D = 200



Problem with large weights

- A model with large weights is more complex than a model with smaller weights.
- In growing the weights, the model becomes overly specialized to handle the specifics of the examples seen in the training data.
- Large weights make our model unstable: minor variation or statistical noise on the expected inputs will result in large differences in the output.
- We refer to this model as having a *large variance* and a *small bias* (Section 6).
- This can be a sign that the model has *overfit* the training dataset and will likely perform poorly when making predictions on new data.
- A solution to this problem is to update the learning algorithm to *encourage the model to keep the weights small*.
- This is called **weight regularization** and it can be used as a general technique to reduce overfitting and improve model generalization.
- Weight regularization (aka weight decay) are commonly used in training large neural networks to prevent overfitting (Module 5)

Outline

Objectives

Revisit model complexity and overfitting

Ridge regression

Bayesian interpretation of regularized regression

L1-regularized regression

- Algorithm of fitting L_1 regularized regression model

- Group LASSO

- Elastic net

Bias-variance trade-off

Summary

Definition of L_p -norm

L1-norm,

$$||\mathbf{w}||_1 = \sum_{d=1} |w_d|$$

L2-norm,

$$||\mathbf{w}||_2 = \left(\sum_{d=1} w_d^2 \right)^{1/2}$$

L_p -norm,

$$||\mathbf{w}||_p = \left(\sum_{d=1} |w_d|^p \right)^{1/p}$$

p-power of a L_p -norm:

$$||\mathbf{w}||_p^p = \sum_{d=1} |w_d|^p$$

For example, the squared of the L2-norm is:

$$||\mathbf{w}||_2^2 = \left(\sum_{d=1} w_d^2 \right)$$

Ridge regression (aka L2-regularized least squares regression)

To penalize large weights, we add

$$J(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \quad (1)$$

where

- $\|\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\|_2^2$ is the sum of square errors as in the linear regression:

$$\|\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\|_2^2 = \sum_n (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w} - w_0)^2 = (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0)^\top (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0)$$

- $\|\mathbf{w}\|_2^2$ is the squared *L2-norm* of the weights:

$$\|\mathbf{w}\|_2^2 = \sum_d w_d^2 = \mathbf{w}^\top \mathbf{w}$$

- λ controls the strength of the regularization on the weights – the larger the lambda the higher the penalty
- λ is a hyper-parameter that requires validation set to choose
- We don't penalize the bias term w_0 .

Recall in Module 4.1: OLS derivation for the bias term w_0

Let the loss function be $J(\mathbf{w}, w_0) = (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1})^\top (\mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1})$.

$$\begin{aligned}\frac{\partial J(\mathbf{w})}{\partial w_0} &= \frac{\partial}{\partial w_0} ((\mathbf{y} - \mathbf{X}\mathbf{w}) - \mathbf{1}w_0)^\top ((\mathbf{y} - \mathbf{X}\mathbf{w}) - \mathbf{1}w_0) \\&= \frac{\partial}{\partial w_0} \{ (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) - (\mathbf{y} - \mathbf{X}\mathbf{w})^\top \mathbf{1}w_0 - w_0 \mathbf{1}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \mathbf{1}^\top \mathbf{1}w_0^2 \} \\&= \frac{\partial}{\partial w_0} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) - \frac{\partial}{\partial w_0} 2w_0 \mathbf{1}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \frac{\partial}{\partial w_0} Nw_0^2 \\&= 0 - 2\mathbf{1}^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + 2Nw_0 \\&= -2\mathbf{1}^\top \mathbf{y} + 2\mathbf{1}^\top \mathbf{X}\mathbf{w} + 2Nw_0 \\&= -2 \sum_n y^{(n)} + 2(\sum_n \mathbf{x}^{(n)})\mathbf{w} + 2Nw_0 \stackrel{\text{set}}{=} 0\end{aligned}$$

Solving $\frac{\partial J(\mathbf{w})}{\partial w_0} = 0$ for w_0 :

$$\hat{w}_0 = \frac{1}{N} \sum_n y^{(n)} - \frac{1}{N} (\sum_n \mathbf{x}^{(n)})\mathbf{w} \equiv \bar{y} - \bar{\mathbf{x}}\mathbf{w}$$

Plugging in the OLS estimate for the bias term w_0 into the loss

$$\begin{aligned} J(\mathbf{w}, \hat{w}_0) &= \|\mathbf{y} - \mathbf{X}\mathbf{w} - \mathbf{1}\hat{w}_0\|_2^2 \\ &= \|\mathbf{y} - \mathbf{X}\mathbf{w} - \mathbf{1}(\bar{y} - \bar{\mathbf{x}}\mathbf{w})\|_2^2 \\ &= \|(\mathbf{y} - \mathbf{1}\bar{y}) - (\mathbf{X}\mathbf{w} - \mathbf{1}\bar{\mathbf{x}}\mathbf{w})\|_2^2 \\ &= \|(\mathbf{y} - \mathbf{1}\bar{y}) - (\mathbf{X} - \mathbf{1}\bar{\mathbf{x}})\mathbf{w}\|_2^2 \\ &\equiv \|\dot{\mathbf{y}} - \dot{\mathbf{X}}\mathbf{w}\|_2^2 \end{aligned}$$

where both the response and input features are mean-centered such that:

$$\begin{aligned} \dot{y}^{(n)} &= y^{(n)} - \bar{y} \\ \dot{\mathbf{x}}^{(n)} &= \mathbf{x}^{(n)} - \bar{\mathbf{x}} \end{aligned}$$

So in the following derivation for the regularized weights $\mathbf{w}$, we can assume the data are centered: $\dot{\mathbf{X}} \equiv \mathbf{X}$ and $\dot{\mathbf{y}} \equiv \mathbf{y}$ so that $w_0 = 0$.

Fitting ridge regression

For simple regression with one feature (assuming $w_0 = 0$ after centering):

$$\begin{aligned}\frac{\partial J(w)}{\partial w} &= \frac{\partial}{\partial w} \left(\frac{1}{2} \sum_n (y^{(n)} - x^{(n)} w)^2 + \frac{\lambda}{2} w^2 \right) = \frac{1}{2} \sum_n 2(y^{(n)} - x^{(n)} w)(-x^{(n)}) + \lambda w \\ &= -\mathbf{x}^\top \mathbf{y} + \mathbf{x}^\top \mathbf{x} w + \lambda w = \mathbf{x}^\top (\mathbf{x} w - \mathbf{y}) + \lambda w\end{aligned}$$

For multiple regression with D features (assuming $\mathbf{X}$ and $\mathbf{y}$ have been centered):

$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} \left(\frac{1}{2} (\mathbf{y} - \mathbf{X}\mathbf{w})^\top (\mathbf{y} - \mathbf{X}\mathbf{w}) + \frac{\lambda}{2} \mathbf{w}^\top \mathbf{w} \right) \stackrel{*}{=} \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}$$

*Refer to Module 4.1 slide 24 for the intermediate step.

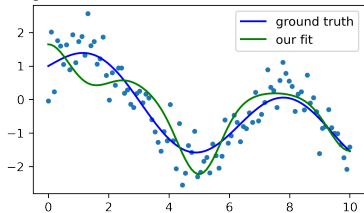
Solving $\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = 0$ for $\mathbf{w}$ gives the closed-form update equation:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

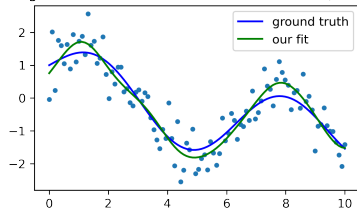
$\lambda \mathbf{I}$ adds a small value to the diagonal elements of $\mathbf{X}^\top \mathbf{X}$, making the matrix inverse numerically stable.

Ridge regression ($\lambda = 0.1$) using 5, 10, 50, 200 Gaussian bases (colab)

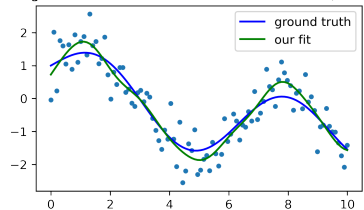
Ridge regression $\lambda = 0.1$ with 5 Gaussian bases (SSE = 43.35)



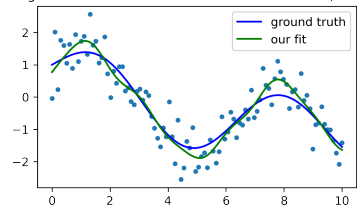
Ridge regression $\lambda = 0.1$ with 10 Gaussian bases (SSE = 21.11)



Ridge regression $\lambda = 0.1$ with 50 Gaussian bases (SSE = 20.72)

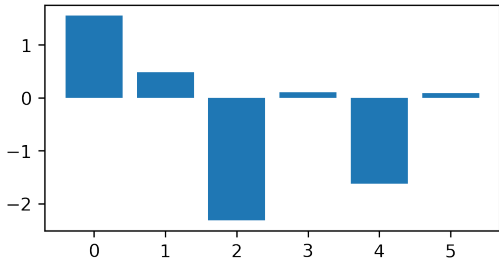


Ridge regression $\lambda = 0.1$ with 200 Gaussian bases (SSE = 20.44)

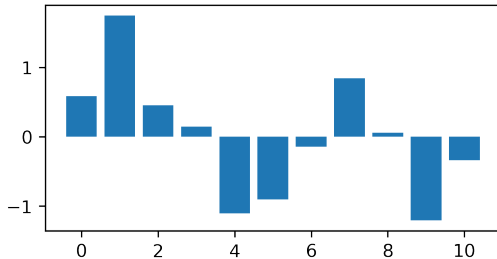


The fitted weights by ridge regression are much more *regularized*

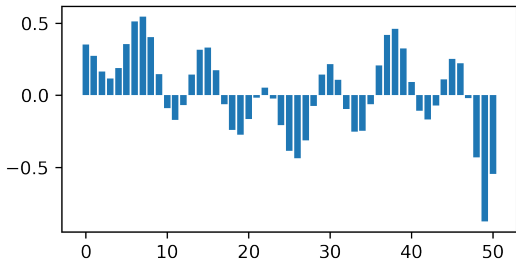
D = 5



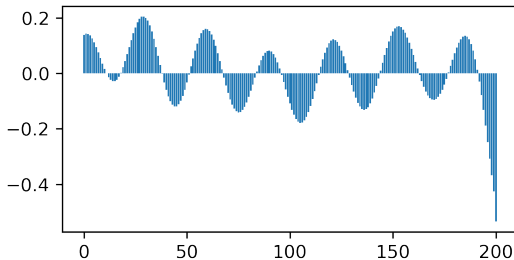
D = 10



D = 50

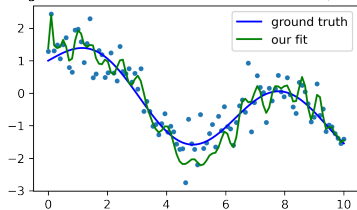


D = 200

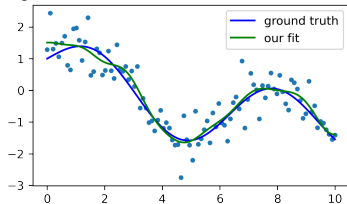


Varying λ for fitting the 200 Gaussian bases

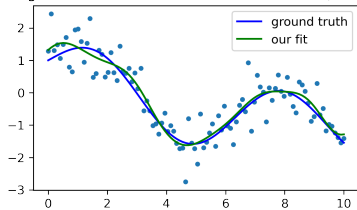
Ridge regression $\lambda = 0$ with 200 Gaussian bases (SSE = 19.44)



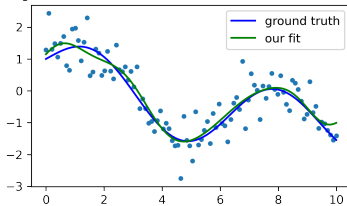
Ridge regression $\lambda = 0.1$ with 200 Gaussian bases (SSE = 19.23)



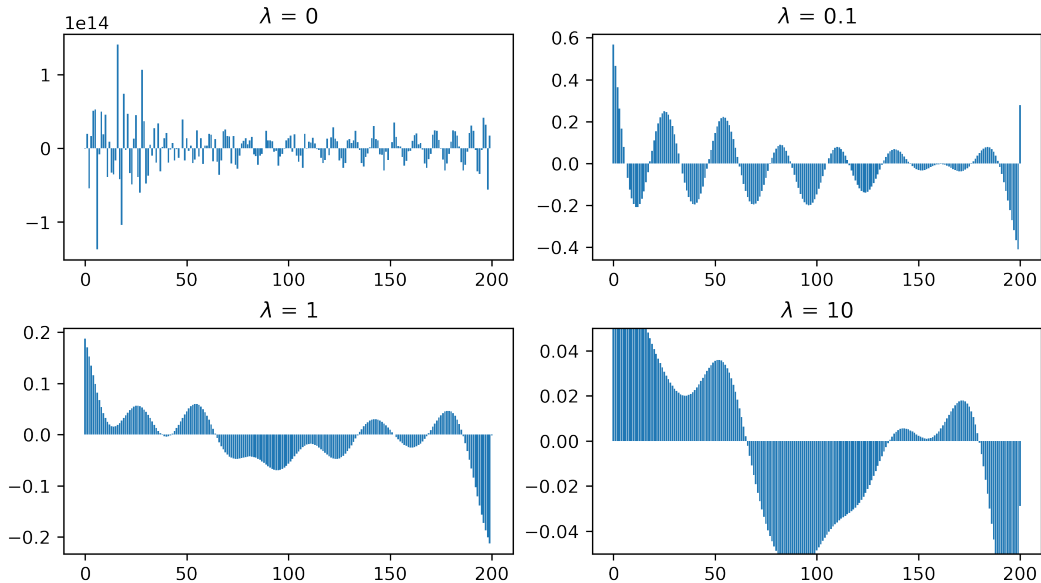
Ridge regression $\lambda = 1$ with 200 Gaussian bases (SSE = 19.96)



Ridge regression $\lambda = 10$ with 200 Gaussian bases (SSE = 20.86)



The fitted weights by ridge regression are much more *regularized*



Outline

Objectives

Revisit model complexity and overfitting

Ridge regression

Bayesian interpretation of regularized regression

L1-regularized regression

- Algorithm of fitting L_1 regularized regression model

- Group LASSO

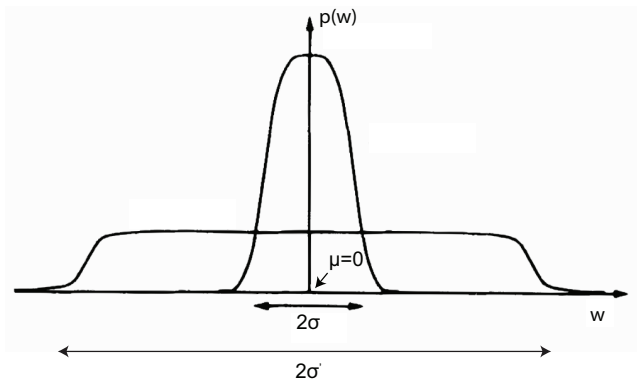
- Elastic net

Bias-variance trade-off

Summary

Gaussian prior

$$p(w) = \mathcal{N}(w; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(w - \mu)^2}{2\sigma^2}\right)$$



Which Gaussian prior encourages large effect size?

Bayesian interpretation of ridge regression

Prior:

$$p(\mathbf{w}) = \prod_d \mathcal{N}(w_d; 0, \sigma_w^2) = \prod_d \frac{1}{\sigma_w \sqrt{2\pi}} \exp\left(-\frac{w_d^2}{2\sigma_w^2}\right)$$

Likelihood:

$$p(\mathbf{y}|\mathbf{x}, \mathbf{w}) = \prod_n \mathcal{N}(y^{(n)}; \mathbf{x}^{(n)}\mathbf{w}, \sigma_y^2) = \prod_n \frac{1}{\sigma_y \sqrt{2\pi}} \exp\left(-\frac{(y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2}{2\sigma_y^2}\right)$$

Posterior:

$$p(\mathbf{w}|\mathbf{y}, \mathbf{x}) = \frac{p(\mathbf{y}|\mathbf{x}, \mathbf{w})p(\mathbf{w})}{p(\mathbf{y}|\mathbf{x})} = \frac{p(\mathbf{y}|\mathbf{x}, \mathbf{w})p(\mathbf{w})}{\int p(\mathbf{y}, \mathbf{w}|\mathbf{x})d\mathbf{w}}$$

In the case of Gaussian prior and Gaussian likelihood, $p(\mathbf{w}|\mathbf{y}, \mathbf{x})$ is also a Gaussian because of the conjugacy of Gaussian to Gaussian (beyond the scope of this course). See my [lecture 4](#) in COMP 565 if you are interested.

Maximum a Posteriori (MAP)

It is often convenient to express log likelihood:

$$\log p(\mathbf{w}|\mathbf{y}, \mathbf{x}) = \log p(\mathbf{y}|\mathbf{x}, \mathbf{w}) + \log p(\mathbf{w}) - \underbrace{\log p(\mathbf{y}|\mathbf{x})}_{\text{constant w.r.t. } \mathbf{w}} \propto \log p(\mathbf{y}|\mathbf{x}, \mathbf{w}) + \log p(\mathbf{w})$$

MAP maximizes the *unnormalized* log joint likelihood $\log p(\mathbf{y}|\mathbf{x}, \mathbf{w})p(\mathbf{w})$ w.r.t. $\mathbf{w}$:

$$\mathbf{w}^* \leftarrow \arg \max_{\mathbf{w}} \log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) + \log p(\mathbf{w})$$

$$\begin{aligned} \log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) &= \log \prod_n \frac{1}{\sigma_w \sqrt{2\pi}} \exp \left(-\frac{(y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2}{2\sigma_y^2} \right) \\ &= \underbrace{\frac{N}{\sigma_y \sqrt{2\pi}}}_{\text{constant w.r.t. } \mathbf{w}} - \sum_n \left(\frac{1}{2\sigma_y^2} (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2 \right) \propto -\frac{1}{2\sigma_y^2} \sum_n (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2 \end{aligned}$$

$$\begin{aligned} \log p(\mathbf{w}) &= \log \prod_d \frac{1}{\sigma_w \sqrt{2\pi}} \exp \left(-\frac{w_d^2}{2\sigma_w^2} \right) = \underbrace{\frac{D}{\sigma_w \sqrt{2\pi}}}_{\text{constant w.r.t. } \mathbf{w}} - \sum_d \left(\frac{w_d^2}{2\sigma_w^2} \right) \propto -\frac{1}{2\sigma_w^2} \|\mathbf{w}\|_2^2 \end{aligned}$$

Connection between MAP and Ridge Regression

Setting $\sigma_y^2 = 1$ and defining $\frac{1}{\sigma_w^2} = \lambda$ leads to the negative cost of ridge regression:

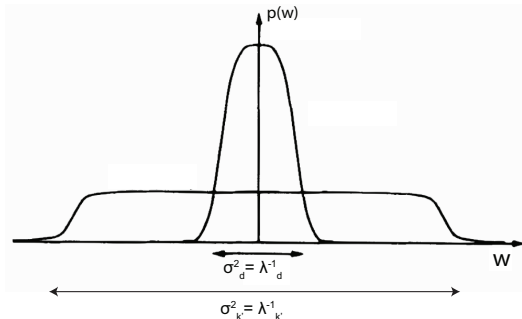
$$\begin{aligned}\log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) + \log p(\mathbf{w}) &\propto -\frac{1}{2\sigma_y^2} \sum_n (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2 - \frac{1}{2\sigma_w^2} \|\mathbf{w}\|_2^2 \\ &\equiv -\frac{1}{2} \sum_n (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2 - \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \\ &= -\left(\frac{1}{2} \sum_n (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2 \right) \\ &= -J_{\text{ridge}}(\mathbf{w})\end{aligned}$$

Note:

- Because $\frac{1}{\sigma_w^2} = \lambda$, we can see the smaller the variance the larger the penalty.
- Here, σ_w^2 is considered as a known constant. In more advanced Bayesian MAP optimization, we can have $\sigma_{w_d}^2$ correspond to each feature d .

Automatic relevance determination (ARD) (optional)

Variable importance is determined by the inverse of their variance ($1/\sigma_{w_d}^2$):



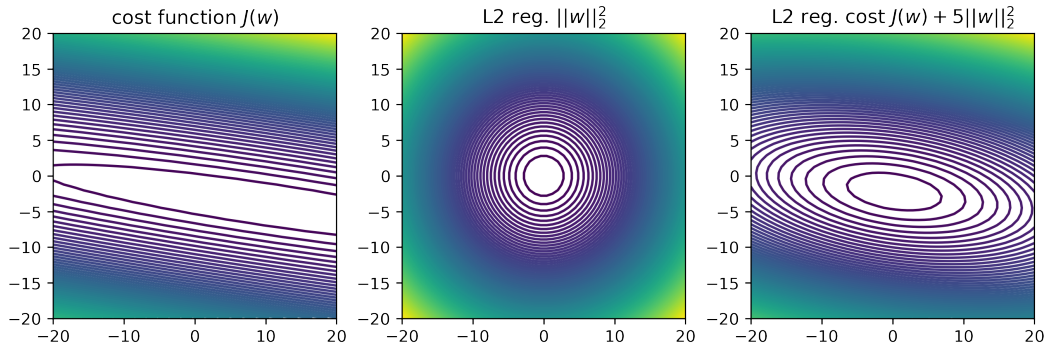
By fitting $\mathbf{w}$ and $\sigma_{\mathbf{w}}^2$ simultaneously we can avoid choosing the hyperparameter λ and let the fitted $\sigma_{w_d}^2$ to automatically determine the relevance of each variable:

$$\mathbf{w}, \sigma_{\mathbf{w}}^2 \leftarrow \arg \max_{\mathbf{w}, \sigma_{\mathbf{w}}^2} - \frac{1}{2\sigma_y^2} \sum_n (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2 - \sum_d \frac{1}{2\sigma_{w_d}^2} w_d^2$$

Visualizing the effect of regularization on the cost function

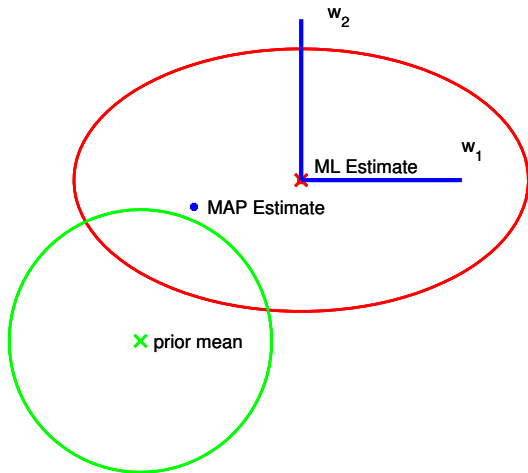
$$J(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

As we see from the shape change of contour in the the 2D weight space, adding the squared L2-norm (middle) to the SSE cost $J(\mathbf{w})$ makes the overall cost more like a circular rings.



Connection between MAP and MLE

In other words, the zero-mean Gaussian prior “pulls” the MLE estimate towards it leading to the MAP estimates between the MLE and the prior.



Outline

Objectives

Revisit model complexity and overfitting

Ridge regression

Bayesian interpretation of regularized regression

L1-regularized regression

- Algorithm of fitting L_1 regularized regression model

- Group LASSO

- Elastic net

Bias-variance trade-off

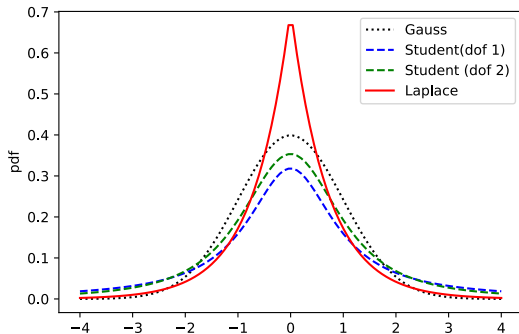
Summary

Laplace prior on variable weights

Instead of having a Gaussian prior on $\mathbf{w}$, we want a more *sparse* prior that “spikes” around zero. Laplace distribution is such a prior:

$$\begin{aligned} p(\mathbf{w}|\lambda) &= \prod_d \text{Lap}(w_d|0, b) \\ &= \prod_d \frac{1}{2b} \exp\left(-\frac{|w_d - \mu|}{b}\right) \\ \log p(\mathbf{w}|\lambda) &= \underbrace{-D \log 2b}_{\text{constant w.r.t. } w_d} - \sum_d \frac{|w_d - \mu|}{b} \\ &\stackrel{\mu=0}{\propto} -\frac{1}{b} \sum_d |w_d| \end{aligned}$$

Therefore, $\log p(\mathbf{w}|\lambda)$ is proportional to the L1-norm $\|\mathbf{w}\|_1$.



MAP regression with Laplace prior and LASSO

$$\begin{aligned}\log p(\mathbf{y}|\mathbf{X}, \mathbf{w}) + \log p(\mathbf{w}) &\propto -\frac{1}{2\sigma_y^2} \sum_n (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2 - \frac{1}{b} \sum_d |w_d| \\ &\stackrel{\sigma_y^2=1, \lambda=1/b}{\equiv} - \left(\frac{1}{2} \sum_n (y^{(n)} - \mathbf{x}^{(n)}\mathbf{w})^2 + \lambda \|\mathbf{w}\|_1 \right) \\ &= -J(\mathbf{w})\end{aligned}$$

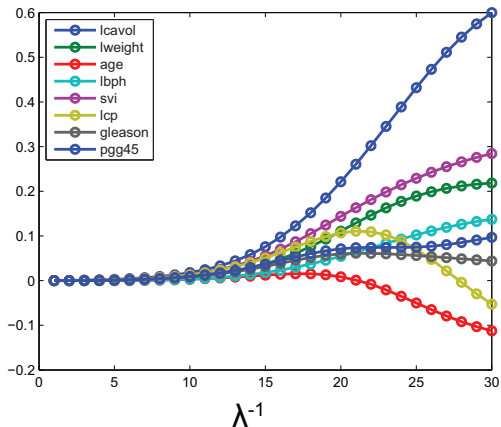
Therefore, MAP with Laplace prior is equivalent to minimizing $J(\mathbf{w})$ with L1-norm weight penalty, which involves a well known technique called Least Absolute Shrinkage and Selection Operator (LASSO) (Tibshirani 1996).

R. Tibshirani. Regression shrinkage and selection via the lasso. In: J. Royal. Statist. Soc B 58.1 (1996), pp. 267–288.

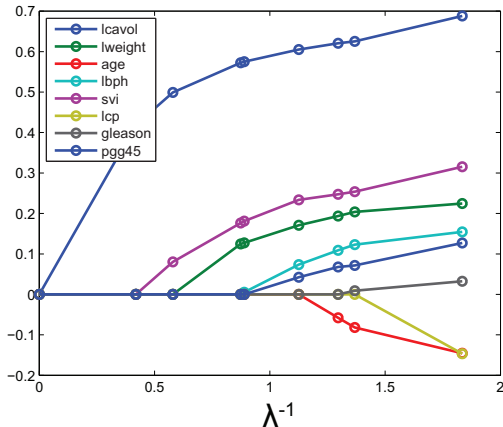
L_2 regularization (Ridge) versus L_1 regularization (LASSO)

The *regularization paths* show how the weights w_d 's change (y-axis) as we decrease λ (x-axis). For instance, for the L_1 regularization path, at $\lambda = 2$, only two variables have non-zero coefficients in the prostate cancer prediction example (i.e., lcvol and svi).

L2-regularized regression (ridge)

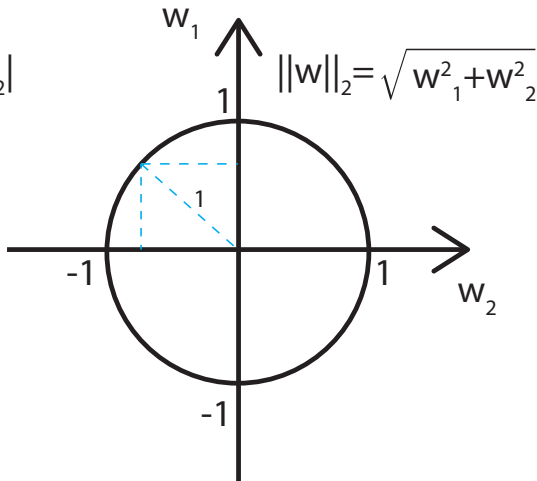
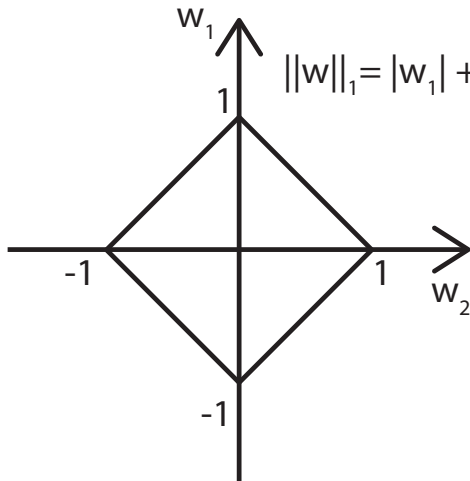


L1-regularized regression (lasso)



Unit ball constraint

Constraint: the L_p -norm $\|\mathbf{w}\|_p = (|w_1|^p + |w_2|^p)^{1/p}$ must be no greater than 1.



In the constrained regression, we replace 1 with λ (next slide).

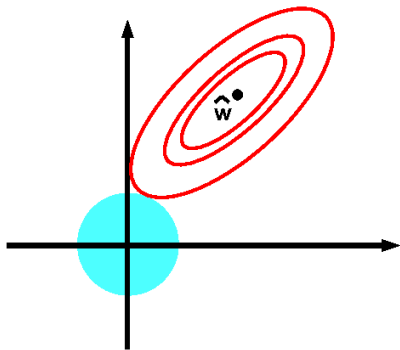
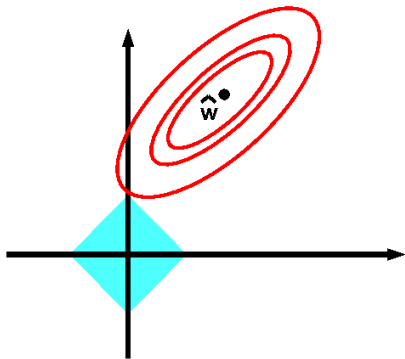
Constrained optimization

Cost function view:

$$\text{LASSO: } J(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_1 \quad \text{Ridge: } J(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

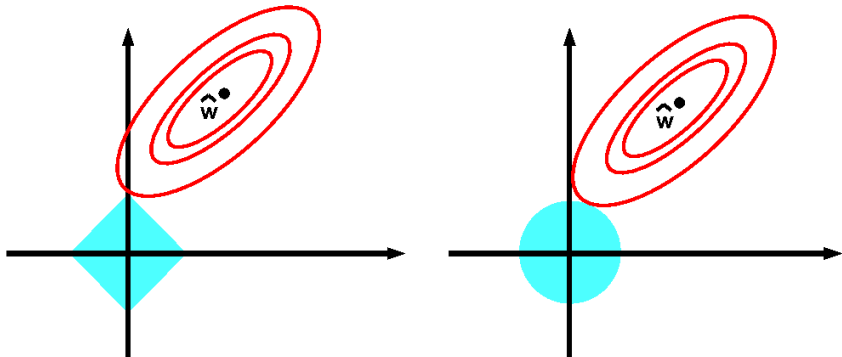
Constrained optimization view:

$$\text{LASSO: } \arg \min \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 \text{ s.t. } \|\mathbf{w}\|_1 \leq \lambda \quad \text{Ridge: } J(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 \text{ s.t. } \|\mathbf{w}\|_2^2 \leq \lambda$$



Why does L1-regularization yield sparse solutions? (Murphy22) (optional)

- The optimal solution occurs at the point where the lowest level set of the objective function intersects the constraint surface
- Geometrically, as we relax the constraint λ , we grow the L_1 “diamond” until it meets the objective
- The corners of the diamond are more likely to intersect the ellipse than one of the sides, especially in high dimensions, because the corners “stick out” more.



Outline

Objectives

Revisit model complexity and overfitting

Ridge regression

Bayesian interpretation of regularized regression

L1-regularized regression

- Algorithm of fitting L_1 regularized regression model

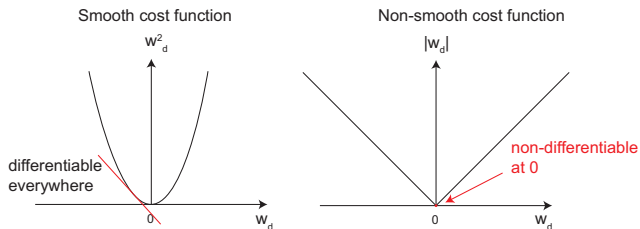
- Group LASSO

- Elastic net

Bias-variance trade-off

Summary

L_1 penalty is a non-smooth but convex function

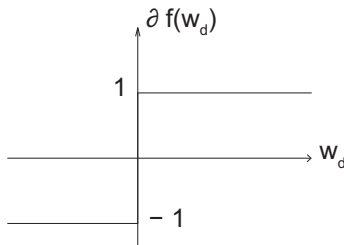
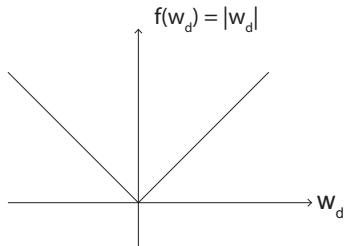
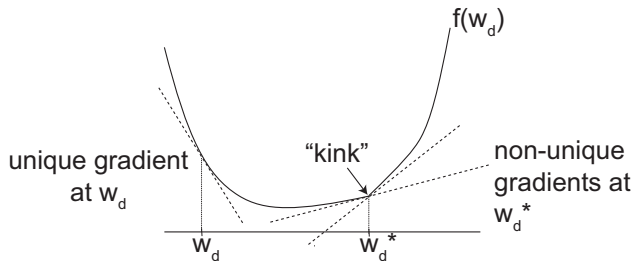


Under L1-norm, the gradient is defined at $w_d \neq 0$ but undefined at $w_d = 0$:

$$\begin{aligned}\|\mathbf{w}\|_1 &= \sum_{d=1}^D |w_d| = \sum_{d=1}^D \sqrt{w_d^2} \\ \frac{\partial \sqrt{w_d^2}}{\partial w_d} &= \frac{\partial \sqrt{w_d^2}}{\partial w_d^2} \frac{\partial w_d^2}{\partial w_d} = \frac{1}{2} \frac{2w_d}{\sqrt{w_d^2}} = \frac{w_d}{\sqrt{w_d^2}} = \begin{cases} \text{sign}(w_d), & w_d \neq 0 \\ \text{undefined}, & w_d = 0 \end{cases}\end{aligned}$$

where $\text{sign}(w_d) = 1$ for $w_d > 0$ and $\text{sign}(w_d) = -1$ for $w_d < 0$.

Subgradient (Murphy22 8.1.4.1)



$$\frac{\partial |w_d|}{\partial w_d} = \begin{cases} -1 & w_d < 0 \\ [-1, 1] & w_d = 0 \\ +1 & w_d > 0 \end{cases}$$

Deriving L_1 -regularized update for w_d while fixing $\mathbf{w}_{\setminus d}$ to $\hat{\mathbf{w}}_{\setminus d}$

Start by differentiating the LASSO objective $J(\mathbf{w}) = ||y - \mathbf{X}\mathbf{w}||_2^2 + \lambda||\mathbf{w}||_1$:

$$\begin{aligned}\frac{\partial J(\mathbf{w})}{\partial w_d} &= \frac{\partial}{\partial w_d} \frac{1}{2} \sum_n (y^{(n)} - \mathbf{x}^{(n)} \mathbf{w})^2 + \lambda ||\mathbf{w}||_1 \\ &= \sum_n (y^{(n)} - \mathbf{x}_{\setminus d}^{(n)} \hat{\mathbf{w}}_{\setminus d} - x_d^{(n)} w_d) (-x_d^{(n)}) + \frac{\partial}{\partial w_d} \lambda |w_d| \\ &= - \sum_n (y^{(n)} - \mathbf{x}_{\setminus d}^{(n)} \hat{\mathbf{w}}_{\setminus d}) x_d^{(n)} + w_d \sum_n (x_d^{(n)})^2 + \frac{\partial}{\partial w_d} \lambda |w_d| \\ &= -\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d ||\mathbf{x}_d||_2^2 + \frac{\partial}{\partial w_d} \lambda |w_d|\end{aligned}$$

Using the notion of subgradient, we can express three case scenarios:

$$\frac{\partial J(\mathbf{w})}{\partial w_d} = \begin{cases} -\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d ||\mathbf{x}_d||_2^2 - \lambda & w_d < 0 \\ [-\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d ||\mathbf{x}_d||_2^2 - \lambda, -\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d ||\mathbf{x}_d||_2^2 + \lambda] & w_d = 0 \\ -\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d ||\mathbf{x}_d||_2^2 + \lambda & w_d > 0 \end{cases}$$

If $w_d < 0$, we can set $\frac{\partial J(\mathbf{w})}{\partial w_d} = 0$ and solve for w_d :

$$-\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d \|\mathbf{x}_d\|_2^2 - \lambda = 0 \implies \hat{w}_d = \frac{\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + \lambda}{\|\mathbf{x}_d\|_2^2}$$

Since it must be true that $\hat{w}_d < 0$ and $\|\mathbf{x}_d\|_2^2 > 0$, we have

$$\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + \lambda < 0 \implies \mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) < -\lambda$$

Similarly, for $w_d > 0$, we have

$$\begin{aligned} -\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + w_d \|\mathbf{x}_d\|_2^2 + \lambda = 0 &\implies \hat{w}_d = \frac{\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) - \lambda}{\|\mathbf{x}_d\|_2^2} > 0 \\ &\implies \mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) > \lambda \end{aligned}$$

Finally, we can derive the following update rules for w_d :

$$\hat{w}_d = \begin{cases} \frac{\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + \lambda}{\|\mathbf{x}_d\|_2^2} & \text{if } \mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) < -\lambda \\ 0 & \text{if } \mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) \in [-\lambda, \lambda] \\ \frac{\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) - \lambda}{\|\mathbf{x}_d\|_2^2} & \text{if } \mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) > \lambda \end{cases}$$

Finally, the update rule for the LASSO algorithm

Programmatically, we can write the update rule using SoftThreshold function:

$$\hat{w}_d = \text{SoftThreshold}(\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}), \lambda) / \|\mathbf{x}_d\|_2^2$$

where

$$\begin{aligned} \text{SoftThreshold}(\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}), \lambda) &= \text{sign}(\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d})) \max(|\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d})| - \lambda, 0) \\ &= \begin{cases} \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) + \lambda & \text{if } \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) < -\lambda \\ 0 & \text{if } \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) \in [-\lambda, \lambda] \\ \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) - \lambda & \text{if } \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}) > \lambda \end{cases} \end{aligned}$$

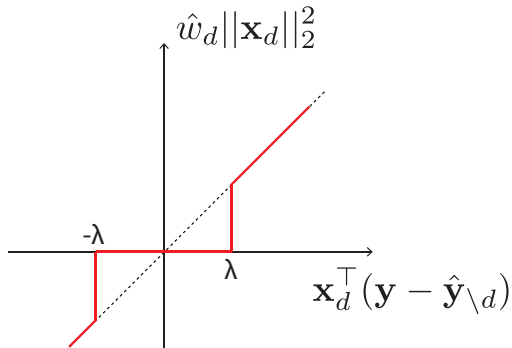
In fact, the soft thresholding implies the **shrinkage** part in the Least Absolute Shrinkage and Selection Operator acronym of LASSO.

Soft-thresholding versus hard-thresholding

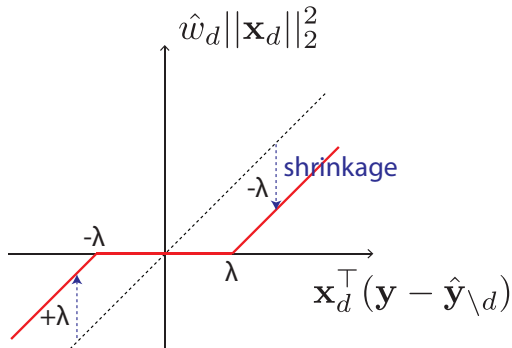
$$\text{HardThreshold}(\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}), \lambda) = \mathbb{I} \left[|\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d})| > \lambda \right] \mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d})$$

$$\text{SoftThreshold}(\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}), \lambda) = \text{sign}(\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d})) \max(|\mathbf{x}_d^\top(\mathbf{y} - \hat{\mathbf{y}}_{\setminus d})| - \lambda, 0)$$

Hard Thresholding



Soft Thresholding



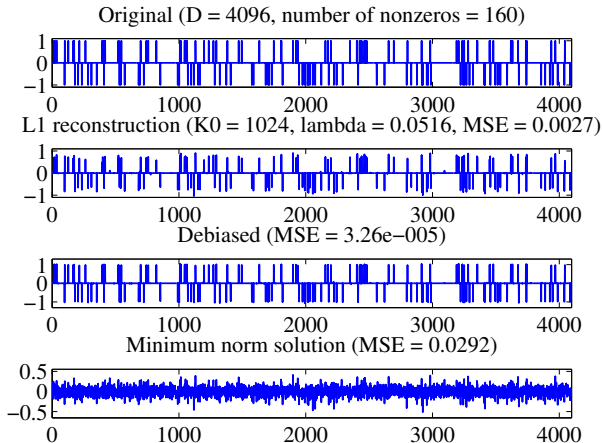
The LASSO *coordinate descent* algorithm

Algorithm 1 LASSO.fit($\mathbf{X}$, $\mathbf{y}$, $\lambda = 0.01$, max_iter=10⁵)

- 1: Randomly initialize regression coefficients $w_d \sim \mathcal{N}(0, 1) \forall d$
 - 2: **for** niter = 1 ... max_iter **do**
 - 3: **for** d = 1 ... D **do**
 - 4: $\hat{w}_d^{\{t\}} \leftarrow \text{SoftThreshold}(\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}), \lambda) / \|\mathbf{x}_d\|_2^2$
 - 5: **end for**
 - 6: **if** $J(\hat{\mathbf{w}}^{\{t\}}) - J(\hat{\mathbf{w}}^{\{t-1\}}) < \epsilon$ **then**
 - 7: break // *Converged so we quite before completing all iterations*
 - 8: **end if**
 - 9: **end for**
-

Two-stage Debiasing: using LASSO for variable selection (Murphy22 11.4.6)

1. Estimate the weights by identifying non-zero weights using LASSO
2. Re-estimate the chosen coefficients using least squares



Simulated data:

- $D = 4096$, 160 non-zero w_d 's, $N = 1024$
- $y = \mathbf{X}\mathbf{w} + \epsilon \sim \mathcal{N}(0, 0.01^2)$

Estimates $\hat{\mathbf{w}}$:

1. True model
2. LASSO estimates
3. De-biased estimates
4. OLS estimates

Group LASSO

Consider the following two scenarios:

- Linear regression with categorical inputs: If the d^{th} variable is categorical with K possible levels, then it will be represented as a one-hot vector of length K , so to exclude variable d , we have to set the whole vector of K weights to 0.
- Multi-class regression: The d^{th} variable is associated with C different weights, one per class, so to exclude variable d , we have to set its C outgoing weights to 0.
(See Generalized linear model in Murphy22 Chapter 12 (optional reading))

Group LASSO cost function

$$J(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \sum_{g=1}^G \|\mathbf{w}_g\|_2, \quad \text{where} \quad \|\mathbf{w}_g\|_2 = \sqrt{\sum_{d \in S_g} w_d^2}$$

$$\frac{\partial \sqrt{\sum_{d \in S_g} w_d^2}}{\partial w_d} = \frac{\partial}{\partial w_d} \left(\sum_d w_d^2 \right)^{1/2} = \frac{w_d}{\sqrt{\sum_d w_d^2}} \quad (\text{undefined when } \sum_d w_d^2 = 0)$$

Pop quiz: Does it make a difference if we used *squared* L2 norm $\sum_{g=1}^G \|\mathbf{w}_g\|_2^2$ instead?

Elastic net (Ridge and LASSO combined)

Motivations

- Strike a balance between Ridge (non-sparse) and LASSO (sparse) to improve prediction performance
- Identifying hidden groups of variables (i.e., without predefining groups as in the group LASSO)

Elastic net cost function

$$J(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda_1 \|\mathbf{w}\|_2^2 + \lambda_2 \|\mathbf{w}\|_1$$

One last point: Data standardization is important to regularized regression

$$\text{Ridge: } \nabla \mathbf{w} = \mathbf{X}^\top (\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}, \quad \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

$$\text{LASSO: } w_d = \text{SoftThreshold}(\mathbf{x}_d^\top (\mathbf{y} - \hat{\mathbf{y}}_{\setminus d}), \lambda) / \|\mathbf{x}_d\|_2^2$$

- The impact of the penalty depends on the scale of the input features.
- For features with large scale, the regularization penalty has less effect.
- Recall we can standardize each feature d by mean centering and scaling:

$$\tilde{x}_d^{(n)} = \frac{x_d^{(n)} - \bar{x}_d}{sd(x_d)}$$

where $\bar{x}_d = \frac{1}{N} \sum_n x_d^{(n)}$ and $sd(x_d) = \sqrt{\frac{1}{N} \sum_n (x_d^{(n)} - \bar{x}_d)^2}$.

- Standardizing the input features ensures that the penalty has the same effect on all input features

Pop quiz: after standardization, what's the value for $\|\tilde{\mathbf{x}}_d\|_2^2$?

Outline

Objectives

Revisit model complexity and overfitting

Ridge regression

Bayesian interpretation of regularized regression

L1-regularized regression

- Algorithm of fitting L_1 regularized regression model

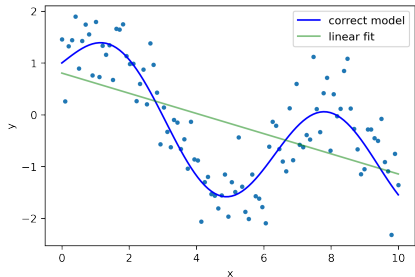
- Group LASSO

- Elastic net

Bias-variance trade-off

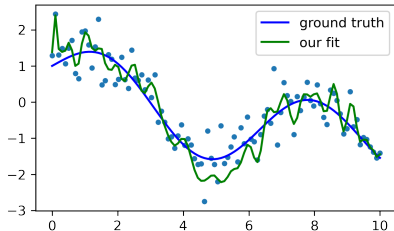
Summary

Generalization and model complexity

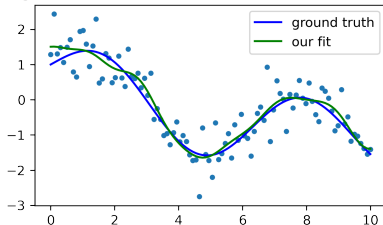


- **Bias:** simple models *underfits* – large training error, away from the true (unknown) function
- **Variance:** complex models *overfits* – small training error, but sensitive to small change of input, large test error and therefore high variance across test data points
- Regularization can help trade-off between bias and variance
- Question: how do bias and variance contribute to the generalization error?

Ridge regression $\lambda = 0$ with 200 Gaussian bases (SSE = 19.44)



Ridge regression $\lambda = 0.1$ with 200 Gaussian bases (SSE = 19.23)



Bias-variance decomposition: setup

- Goal: decompose generalization error in terms of the L2 loss
- Dataset: $\mathcal{D} = \{(x^{(n)}, y^{(n)})\}_{n=1}^N$ is sampled from $p(\mathbf{x}, y)$
- $\hat{f}_{\mathcal{D}}$ be the trained function based on dataset $\mathcal{D}$ (e.g., linear regression)
- The generalization error (aka expected loss) is the expectation over *all datasets*:

$$\mathbb{E}_{\mathcal{D}} \left[(\hat{f}_{\mathcal{D}}(\mathbf{x}) - y)^2 \right]$$

where

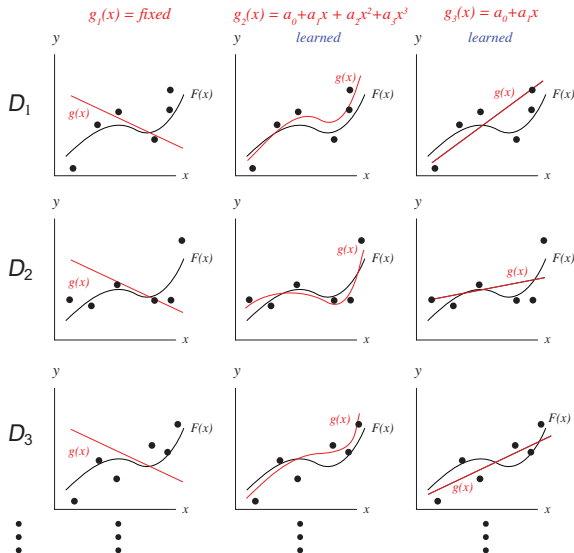
- $\hat{f}_{\mathcal{D}}(\mathbf{x})$ is the predicted value of the target variable (i.e., $\hat{y}$)
- The true value y can be decomposed into the true function output plus the noise:

$$y = f(\mathbf{x}) + \epsilon$$

Bias-variance decomposition: derivation

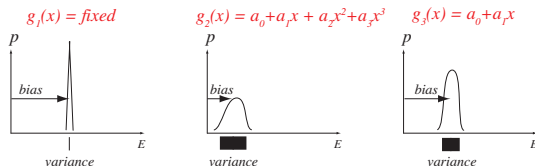
$$\begin{aligned}\mathbb{E}_{\mathcal{D}} \left[(\hat{f}(\mathbf{x}; \mathcal{D}) - y)^2 \right] &= \mathbb{E}_{\mathcal{D}} \left[(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] + \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - y)^2 \right] \\&= \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] \right)^2 + 2(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})])(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - y) + \left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - y \right)^2 \right] \\&= \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] \right)^2 \right] + \underbrace{\mathbb{E}_{\mathcal{D}} \left[2(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})])(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - y) \right]}_0 + \mathbb{E}_{\mathcal{D}} \left[\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - y \right)^2 \right] \\&= \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] \right)^2 \right] + \mathbb{E}_{\mathcal{D}} \left[\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x}) - \epsilon \right)^2 \right] \quad (y = f(\mathbf{x}) + \epsilon) \\&= \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] \right)^2 \right] + \mathbb{E}_{\mathcal{D}} \left[\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x}) \right)^2 \right] - \underbrace{\mathbb{E}_{\mathcal{D}} \left[2\epsilon \left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x}) \right) \right]}_0 + \mathbb{E}_{\mathcal{D}}[\epsilon^2] \\&= \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] \right)^2 \right]}_{\text{Variance}} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x}) \right)^2 \right]}_{\text{Bias}} + \underbrace{\mathbb{E}_{\mathcal{D}}[\epsilon^2]}_{\text{noise}}\end{aligned}$$

Evaluating generalization error of a model using multiple datasets



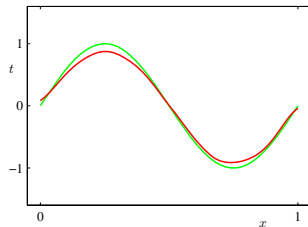
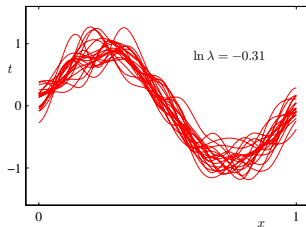
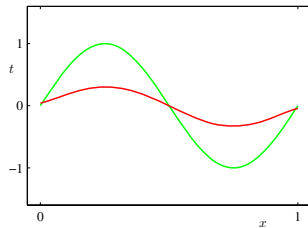
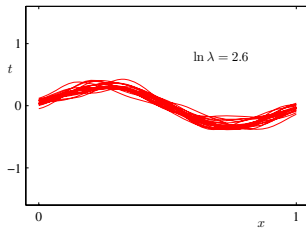
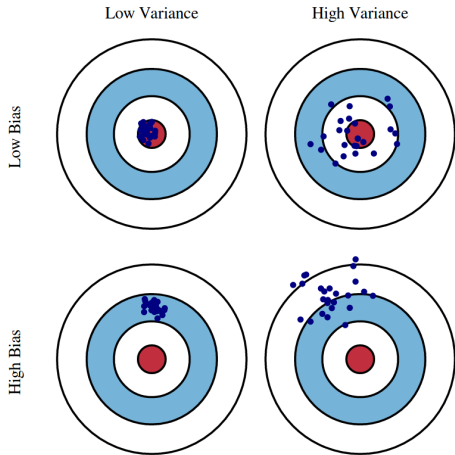
- rows: different datasets; columns: different model $g(x)$
- black curve: the true function
- $g_1(x)$ has low variance but high bias
- $g_2(x)$ has high variance but low bias
- $g_3(x)$ has lower variance than $g_2(x)$ but higher bias

Duda, R. O., Hart, P. E., & Stork, D. G. (2000).
Pattern classification 2nd Ed.



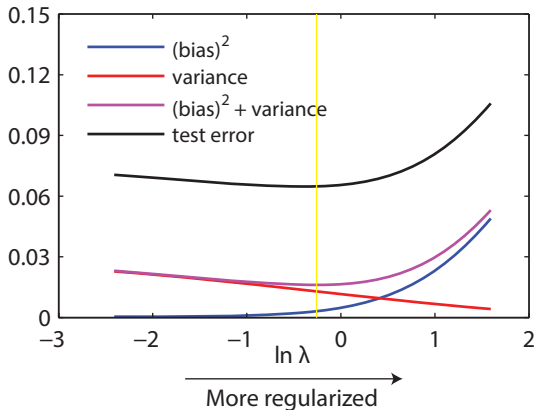
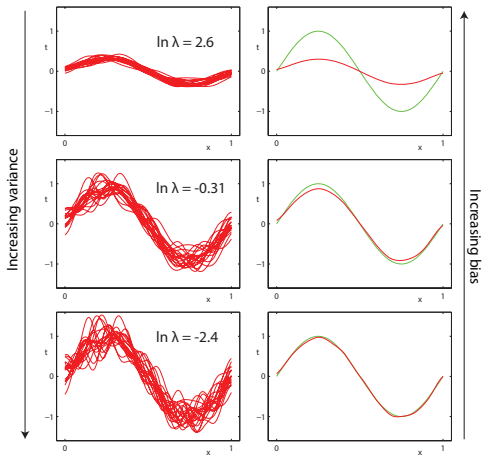
Bias-variance trade-off

Ideal model has low bias, low variance; simple model has high bias, low variance; complex model has low bias, high variance; random model has high bias, high variance.



Bias-variance trade-off

$$\mathbb{E}_{\mathcal{D}} \left[(\hat{f}(\mathbf{x}; \mathcal{D}) - y)^2 \right] = \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] \right)^2 \right]}_{\text{Variance}} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x}) \right)^2 \right]}_{\text{Bias}} + \underbrace{\mathbb{E}_{\mathcal{D}} [\epsilon^2]}_{\text{noise}}$$



The Bayesian way (optional)

- Above frequentist's view of bias-variance tradeoff theory is elegant but may have limited practical value. However, we see the benefits of *model averaging*.
- If we do have multiple datasets, instead of training multiple models one per dataset and averaging over their outputs $\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})]$, we would just combine them and train a single model.
- The Bayesian way of handling the problem is to *average over models* given a single training dataset $\mathcal{D}$, e.g.,

$$\hat{y}^* = \mathbf{x}^* \mathbb{E}_{\mathbf{w}}[\mathbf{w}; \mathcal{D}] = \mathbf{x}^* \int p(\mathbf{w}|\mathcal{D}) \mathbf{w} d\mathbf{w} \approx \mathbf{x}^* \frac{1}{S} \sum_{s=1}^S \hat{\mathbf{w}}$$

where

- $p(\mathbf{w}|\mathcal{D})$ is the posterior distribution
- When $p(\mathbf{w}|\mathcal{D})$ is intractable, we can average the model by sampling $\hat{\mathbf{w}}$ S times from an approximate distribution that is proportional to $\log p(\mathbf{y}|\mathbf{x}, \mathbf{w}) + \log p(\mathbf{w})$. Common procedures include Importance Sampling, Gibbs sampling, Metropolis-Hastings (Murphy12 23.4, 24.1, 24.3, respectively).

Summary

- Overfitting occurs for large number of features relative to the sample size
- L_1 and L_2 regularization to account for overfitting
- Bayesian interpretation and MAP: L_2 : Gaussian prior; L_1 : Laplace prior
- L_1 produces sparse solutions (useful for feature selection)
- Fitting LASSO algorithm by cycling through each feature and updating them using soft-threshold function (i.e., coordinate descent)
- Common model variants: Group LASSO, Elastic net
- Bias-variance trade-off formalizes the relation between training error (bias) and complexity (variance) and the test error (bias + variance)